

Deklaracja projektowa		
Temat projektu:	CourseManager - system zapisów na kursy	
Github	https://github.com/karlosiwo/CourseManager	
Skład grupy		
Imię i nazwisko	Nr indexu	PS
Michał Szwabowicz	111112	PS7
Jakub Wierciszewski	117268	PS7
Karol Ziemak	117292	PS7
Zakres funkcjonalności		
MVP(Must Have Features)	Nice to Have Features	
• Strona powitalna oraz rejestracja użytkownika • Logowanie i wylogowanie użytkownika • Role: admin, użytkownik z ograniczonymi prawami, użytkownik z pełnymi prawami oraz użytkownik niezalogowany • Lista dostępnych kursów oraz szczegóły kursu • Dodanie, edycja i usunięcie własnego zapisu na kurs przez użytkownika z pełnymi prawami • Kontrola limitu miejsc oraz blokada podwójnego zapisu na ten sam kurs • Lista własnych zapisów i zapisów udostępnionych użytkownikowi • Sortowanie kursów w obu kierunkach według daty, nazwy i kategorii • Filtrowanie kursów według daty oraz kategorii • Panel administratora: lista użytkowników i zarządzanie rolami • Formularze z walidacją danych wejściowych • Baza danych z co najmniej dwiema tabelami powiązanymi relacją • Spring Security z użytkownikami zapisanymi w bazie danych • Sesja, ciasteczka, usługa REST oraz klient REST	• Raport najpopularniejszych kursów • Historia zapisów i anulowań • Status zapisu, np. AKTYWNY / ANULOWANY • Raport kursów z największą liczbą wolnych miejsc • Eksport listy uczestników kursu do pliku CSV/PDF • Powiadomienie o braku wolnych miejsc • Bootstrap i poprawiony wygląd interfejsu zgodny z WCAG 2.1 • Dodatkowy widok statystyk aktywności użytkownika	

Do czego REST będzie używane i dlaczego jego użycie ma sens w kontekście projektu.
• REST będzie wykorzystany do weryfikacji nazwy kategorii kursu w słowniku dozwolonych kategorii • REST będzie służył do zwracania liczby wolnych miejsc na kursie w formacie JSON • Aplikacja sprawdzi, czy kategoria istnieje w słowniku i spełnia wymagania nazwy za pomocą GET/api/categories/validate?name=java • Aplikacja zwróci liczbę wolnych miejsc na kursie za pomocą GET /api/courses/{id}/availability • Klient REST po stronie aplikacji lub widoku będzie pobierał te dane i wyświetlał komunikaty użytkownikowi bez konieczności przeładowania całej strony
Jakie encje będą obsługiwane przez operacje CRUD oraz jak pasuje to do projektu.
• Użytkownik: Rejestracja, wyświetlanie listy użytkowników przez administratora, edycja danych, zarządzanie rolami • Kurs: Dodawanie, edycja, usuwanie i wyświetlanie kursów oraz filtrowanie i sortowanie listy kursów • Zapis: Utworzenie zapisu, anulowanie zapisu, sprawdzanie statusu, lista własnych zapisów oraz lista uczestników kursu • Kategoria: Dodanie nowej kategorii, weryfikacja nazwy przez REST, przypisywanie kategorii do kursu • Prowadzący: Dodawanie i edycja prowadzących oraz przypisywanie prowadzącego do kursu
Jakie będą mechanizmy obsługi formularzy oraz walidacji danych wejściowych użytkownika.
• Zostaną przygotowane formularze m.in. do rejestracji, logowania, dodawania/edycji kursu, zapisu na kurs, anulowania zapisu, dodawania kategorii oraz filtrowania listy kursów • Walidacja będzie realizowana po stronie klienta oraz po stronie serwera z użyciem mechanizmów Spring Validation, np. @NotBlank, @Size, @Email, @Min i @Pattern • Formularze będą wykorzystywać co najmniej trzy różne typy pól, np. input text, password, select, date, textarea i checkbox
Przewidywane trudności algorytmiczne:
• Poprawne połączenie Spring Security z bazą danych i obsługa ról użytkowników • Kontrola limitu miejsc, szczególnie w sytuacji, gdy kilku użytkowników próbuje zapisać się na ten sam kurs • Blokowanie podwójnych zapisów tego samego użytkownika na ten sam kurs • Zapis zmian do bazy danych przy wylogowaniu lub wygaśnięciu sesji • Zapamiętanie oraz odtworzenie kryteriów sortowania i filtrowania • Przygotowanie usługi REST i klienta REST w sposób zgodny z wymaganiami projektu • Przygotowanie czytelnej dokumentacji wskazującej, w jakich plikach zrealizowano elementy punktowane